Overview
Welcome to Semantica

Core Concepts
Core Concepts
Modules
Choose the Right Module
Glossary

Guides
Architecture
Learning More
GraphRAG: Graph-Augmented Retrieval
Ingest Anything
LLM Integrations
Pipeline Builder
Multi-Agent Systems
Decision Intelligence
Context Graphs
Agent Memory
Provenance & Audit Trails
Semantic Extraction
Ontology Management
Reasoning & Rules
Export & Serialization
Deduplication & Entity Merging
MCP Server
SHACL Validation
Conflict Detection & Resolution
Change Management & Versioning
Policy Engine
Visualization
Distance Intelligence
Graph Analytics

Integrations
Agno Integration
CrewAI Integration
LangChain Integration
Docling Integration
Snowflake Integration
Databricks Integration
Salesforce Integration

Graph Stores
Apache AGE Graph Store

Vector Stores
pgvector Store

FAQ
FAQ

Community
Community
Community Projects
Contributing
Governance
Citation
License

On this page
Architecture Overview
Input Layer
Ingest
Parse
Split
Normalize
Core Processing
Semantic Extract
Knowledge Graph
Ontology
Reasoning
Storage
Embeddings
Vector Store
Graph Store
Triplet Store
Quality Assurance
Deduplication
Conflicts
Context & Memory
Context
Provenance
Change Management
Output & Orchestration
Export
Visualization
Pipeline
Explorer
Utilities
LLM Providers
MCP Server
Seed
Evals
Core
Utils
Common Module Chains
Module Index

Core Concepts

Modules

Every Semantica module works independently: use only what you need.

Jump to the Module Index for a quick reference.

The Choose the Right Module guide maps 35+ developer goals to modules with code examples; start there if you're orienting for the first time.

Semantica is organized into **27 modules** across six logical layers. Each module is independently importable: you never pay for what you don't use.

Architecture Overview

- **Input Layer**: data ingestion and preparation. Modules: `ingest`, `parse`, `split`, `normalize`
- **Core Processing**: intelligence and understanding. Modules: `semantic_extract`, `kg`, `ontology`, `reasoning`
- **Storage**: persistent data storage. Modules: `embeddings`, `vector_store`, `graph_store`, `triplet_store`
- **Quality Assurance**: data quality and consistency. Modules: `deduplication`, `conflicts`
- **Context & Memory**: agent memory and decision tracking. Modules: `context`, `provenance`, `change_management`
- **Output & Orchestration**: export, visualization, and workflows. Modules: `export`, `visualization`, `pipeline`, `explorer`

Input Layer

Ingest

Loads data from files, web, databases, and streams. Each ingestor returns its own result type (`FileIngestor` → `FileObject`, `WebIngestor` → `WebContent`, ...); document-oriented ones expose a `.text` payload and `.metadata`.

```
from semantica.ingest import FileIngestor, WebIngestor, ParquetInge

# Files: PDF, DOCX, CSV, Excel, PPTX, JSON, HTML, archives
ingestor = FileIngestor()
documents = ingestor.ingest_directory("data/")

# Web page: returns a WebContent with .text, .title, .links, .metadata
web_ingestor = WebIngestor()
page = web_ingestor.ingest_url("https://example.com")

# Parquet: single file, partitioned directory, Hive-style (v0.5.0)
parquet = ParquetIngestor()
sources = parquet.ingest("data/events.parquet")

# XML with XSD/DTD validation, namespace handling (v0.5.0)
xml = XMLIngestor()
sources = xml.ingest("data/records/", schema_path="schema.xsd")

# Enterprise lakehouse/warehouse: Unity Catalog + Delta Lake, or a Snowflake wa
databricks = DatabricksIngestor(host="...", token="...", http_path="...")
customers   = databricks.ingest_table("customers")
```

Available ingestors: `FileIngestor`, `WebIngestor`, `ParquetIngestor`, `XMLIngestor`, `RESTIngestor`, `PublicAPIIngestor`, `DBIngestor`, `DatabricksIngestor`, `SnowflakeIngestor`, `EmailIngestor`, `FeedIngestor`, `MCPIngestor`, `OntologyIngestor`, `RepoIngestor`, `StreamIngestor`, `ArrowIngestor`, `CloudStorageIngestor`

`DuckDBIngestor`, `ElasticIngestor`, `GDriveIngestor`, `HuggingFaceIngestor`, `MongoIngestor`, and `PandasIngestor` also ship but aren't re-exported from the top-level `semantica.ingest` namespace yet; import them directly, e.g. `from semantica.ingest.duckdb_ingestor import DuckDBIngestor`.

Parse

Extracts structured text and layout metadata from raw documents.

```
from semantica.parse import DocumentParser, DoclingParser

# Standard parser: all common formats. parse() takes a path, returns a dict
parser = DocumentParser()
parsed = parser.parse("document.pdf")   # {"full_text": ..., "metadata": ...,

# Advanced parser (pip install semantica[parse-docling]): tables, OCR, layout
parser = DoclingParser(export_format="markdown", enable_ocr=True)
parsed = parser.parse("data/annual_report.pdf")   # dict with full_text, tables
```

Available parsers: `DocumentParser`, `DoclingParser`, `CodeParser`, `CSVParser`, `DocxParser`, `EmailParser`, `ExcelParser`, `HTMLParser`, `ImageParser`, `JSONParser`, `MCPParser`, `MediaParser`, `PDFParser`, `PPTXParser`, `StructuredDataParser`, `WebParser`, `XMLParser`

Split

Chunks text for embedding and RAG pipelines with awareness of semantic boundaries.

```
from semantica.split import TextSplitter

# chunk_size / chunk_overlap are constructor arguments
splitter = TextSplitter(method="semantic_transformer", chunk_size=1000, chunk_c
chunks = splitter.split(text)
```

Chunking methods: `recursive`, `token`, `sentence`, `paragraph`, `semantic_transformer`, `entity_aware`, `relation_aware`, `graph_based`, `ontology_aware`, `hierarchical`, `community_detection`, `centrality_based`, `llm`

Normalize

Cleans and standardizes text before semantic processing.

```
from semantica.normalize import TextNormalizer, normalize_text, nor

normalizer = TextNormalizer()
clean_text        = normalizer.normalize_text(text)
standardized_date = normalize_date("Jan 1st, 2020")
```

Normalizers available: text cleaning, entity canonicalization, date normalization, number normalization, encoding handling, language detection

Core Processing

Semantic Extract

Named entity recognition, relation extraction, and triplet generation.

```
from semantica.semantic_extract import NERExtractor, RelationExtrac

# LLM method: provider + llm_model select the backend; the API key comes from t
ner = NERExtractor(method="llm", provider="groq", llm_model="llama-3.3-70b-vers
entities = ner.extract("Apple Inc. was founded by Steve Jobs.")   # list[Entity

rel = RelationExtractor(method="llm", provider="groq", llm_model="llama-3.3-70b
relationships = rel.extract(text, entities=entities)              # list[Relati

trip = TripletExtractor(method="pattern")
triplets = trip.extract(text)                                     # list[Triple
```

Extraction methods: `"pattern"` (no API key), `"ml"` (local spaCy model), `"llm"` (any of the 9 supported providers)

Additional extractors: `CoreferenceResolver`, `EventDetector`, `SemanticAnalyzer`, `SemanticNetworkExtractor`

Knowledge Graph

Graph construction, graph algorithms, temporal model, and distance intelligence.

```
from semantica.kg import GraphBuilder, GraphAnalyzer, TemporalGraph
from datetime import datetime

# Build: build() takes a {"entities": ..., "relationships": ...} dict
builder = GraphBuilder(merge_entities=True)
kg = builder.build({"entities": entities, "relationships": relationships})

# Temporal graphs (v0.4.0)
query_engine = TemporalGraphQuery(enable_temporal_reasoning=True)
snapshot = query_engine.query_at_time(kg, query="", at_time=datetime(2021, 6, 1

# Semantic similarity (v0.5.0): operates on embedding vectors
calc = SimilarityCalculator(method="cosine")
score = calc.cosine_similarity(vec_a, vec_b)
```

Graph algorithms available: centrality calculation, community detection, connectivity analysis, entity resolution, link prediction, path finding, similarity calculation

Ontology

Schema management including SHACL, SKOS, alignments, diff/migration, auto-generation, and the visual Ontology Hub (v0.5.0).

```
from semantica.ontology import OntologyGenerator, SHACLGenerator

generator = OntologyGenerator()
ontology  = generator.generate_from_graph(kg)

shacl  = SHACLGenerator()
shapes = shacl.generate(ontology)
```

Components: `OntologyGenerator`, `SHACLGenerator`, `OntologyValidator`, `OntologyEvaluator`, `LLMOntologyGenerator`, `OWLGenerator`, `PropertyGenerator`, `DomainOntologies`, `NamespaceManager`

Reasoning

Derives new facts from existing knowledge using multiple inference strategies.

```
from semantica.reasoning import Reasoner, DatalogReasoner

# Forward chaining: facts and rules as predicate(args) / IF-THEN strings
engine = Reasoner()
engine.add_fact("Manager(Alice)")
engine.add_rule("IF Manager(?x) THEN HasAuthority(?x)")
results = engine.forward_chain()          # list[InferenceResult] with .conclu

# Datalog: recursive Horn clause rules (v0.4.0)
datalog = DatalogReasoner()
datalog.add_fact("parent(tom, bob)")
datalog.add_fact("parent(bob, ann)")
datalog.add_rule("ancestor(X, Y) :- parent(X, Y).")
datalog.add_rule("ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).")
datalog.derive_all()
results = datalog.query("ancestor(tom, ?Z)")   # [{"Z": "bob"}, {"Z": "ann"}],
```

Engines: `Reasoner` (forward/backward chaining), `ReteEngine`, `SPARQLReasoner`, `DatalogReasoner`, `TemporalReasoningEngine`, `GraphReasoner` (LLM)

Storage

Embeddings

Generates and manages vector embeddings for semantic similarity.

```
from semantica.embeddings import EmbeddingGenerator

generator  = EmbeddingGenerator()
embeddings = generator.generate_embeddings(["text1", "text2"])   # np.ndarray
similarity = generator.compare_embeddings(embeddings[0], embeddings[1])
```

Supported models: Sentence-Transformers, FastEmbed, OpenAI, BGE

Components: `EmbeddingGenerator`, `TextEmbedder`, `VectorEmbeddingManager`, `GraphEmbeddingManager`, `PoolingStrategies`

Vector Store

Multi-backend vector database with hybrid search support.

```
from semantica.vector_store import VectorStore

store = VectorStore(backend="faiss", dimension=768)

# Raw vectors
ids     = store.store_vectors(embeddings)                # returns generated i
hits    = store.search_vectors(query_vector, k=10)

# Or store text and let the store embed it
store.add_documents(["Apple was founded in 1976.", "Google was founded in 1998.
results = store.search("tech company founding dates", limit=10)
```

Backends: FAISS, Pinecone, Weaviate, Qdrant, Milvus, PgVector, SQLite, in-memory

Search modes: semantic top-k, hybrid (vector + keyword), metadata-filtered

Graph Store

Connects to graph databases for persistent, query-able storage.

```
from semantica.graph_store import GraphStore

store = GraphStore(backend="neo4j")
store.add_nodes([{"id": "acme", "type": "Organization", "properties": {"name":
store.add_edges([{"source": "alice", "target": "acme", "type": "works_for"}])
results = store.query("MATCH (n)-[r]->(m) RETURN n, r, m")
```

Backends: Neo4j, FalkorDB, Apache AGE, Amazon Neptune

Triplet Store

RDF triple-based storage with SPARQL query support.

```
from semantica.triplet_store import TripletStore

store = TripletStore(backend="oxigraph")
store.add_triplets(triplets)                 # list of Triplet objects (or add_
results = store.execute_query("SELECT ?s ?p ?o WHERE { ?s ?p ?o }")
```

Backends: Oxigraph (embedded), Blazegraph, Apache Jena, RDF4J

Quality Assurance

Deduplication

Detects, scores, and merges duplicate entities across sources.

```
from semantica.deduplication import DuplicateDetector, EntityMerger

detector   = DuplicateDetector(similarity_threshold=0.85)
candidates = detector.detect_duplicates(entities)

merger     = EntityMerger()
operations = merger.merge_duplicates(entities, strategy="keep_most_complete")
```

v2 candidate-generation modes (`blocking_v2`, `hybrid_v2`, `semantic_v2`) are up to 7x faster than v1.

Components: `DuplicateDetector`, `EntityMerger`, `ClusterBuilder`, `MergeStrategyManager`

`DuplicateDetector` **options:** `max_results`, `top_k_per_entity`, `min_similarity`, `sort_by`

Conflicts

Detects and resolves fact conflicts across overlapping knowledge sources.

```
from semantica.conflicts import ConflictDetector, ConflictResolver

conflicts = ConflictDetector().detect_conflicts(entities)   # list of entity di
resolved  = ConflictResolver().resolve_conflicts(conflicts, strategy="most_rece
```

Detection types: value conflicts, type conflicts, relationship conflicts, temporal conflicts, logical conflicts

Resolution strategies: prefer most recent, prefer most reliable source, majority vote, flag for manual review

Context & Memory

Context

Agent context graphs, decision tracking, causal chains, and precedent search.

```
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore

context = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=ContextGraph(advanced_analytics=True),
    decision_tracking=True,
)

context.store("GPT-4 outperforms GPT-3.5 on reasoning benchmarks by 40%")

decision_id = context.record_decision(
    category="model_selection",
    scenario="...",
    reasoning="...",
    outcome="...",
    confidence=0.9,
)

precedents = context.find_precedents("model selection", limit=5)
```

Components: `AgentContext`, `ContextGraph`, `AgentMemory`, `DecisionRecorder`, `CausalAnalyzer`, `EntityLinker`, `PolicyEngine`

Provenance

W3C PROV-O compliant lineage tracking across all modules.

```
from semantica.provenance import ProvenanceManager

manager = ProvenanceManager()
manager.track_entity("entity_1", source="document.pdf", metadata={"type": "pers
lineage = manager.get_lineage("entity_1")
```

Components: `ProvenanceManager`, `IntegrityChecker`, `BridgeAxiom`, `ProvenanceStorage`

Change Management

Version control with SHA-256 checksums, diffs, and rollback.

```
from semantica.change_management import TemporalVersionManager

manager  = TemporalVersionManager(storage_path="versions.db")
snapshot = manager.create_snapshot(kg, "v1.0", "user@example.com", "Initial ver
diff     = manager.diff("v1.0", "v1.1")
```

Components: `TemporalVersionManager`, `ChangeLog`, `OntologyVersionManager`, `VersionStorage`

Output & Orchestration

Export

Serializes graphs to downstream formats for analytics, semantic web, or graph databases.

```
from semantica.export import RDFExporter, ParquetExporter, ArangoAQ

# RDF formats
RDFExporter().export(graph, file_path="graph.ttl", format="turtle")

# Analytics
ParquetExporter().export(graph, file_path="output/graph.parquet")

# ArangoDB: writes AQL INSERT statements to the given path
ArangoAQLExporter().export(graph, file_path="graph.aql")
```

Export formats: RDF (Turtle, JSON-LD, N-Triples, XML), Parquet, ArangoDB AQL, CSV, OWL, Arrow, LPG, YAML, distance matrices

Visualization

Renders interactive and static knowledge graph visualizations.

```
from semantica.visualization import KGVisualizer

viz = KGVisualizer()
viz.visualize_network(graph, output="html", file_path="graph.html")
```

Visualizers: `KGVisualizer`, `OntologyVisualizer`, `EmbeddingVisualizer`, `SemanticNetworkVisualizer`, `TemporalVisualizer`, `AnalyticsVisualizer`

Layout algorithms: force-directed, hierarchical, circular

Pipeline

Pipeline DSL with parallel workers, retry policies, and failure handling.

```
from semantica.pipeline import PipelineBuilder, ExecutionEngine
from semantica.ingest import FileIngestor
from semantica.semantic_extract import NERExtractor

builder = PipelineBuilder()

# Each step type dispatches to a handler you register (or supply explicitly)
builder.register_step_handler("ingest",  lambda data, **c: FileIngestor().inges
builder.register_step_handler("extract", lambda docs, **c: NERExtractor(method=

builder.add_step("ingest",  step_type="ingest", source="data/")
builder.add_step("extract", step_type="extract")

pipeline = builder.connect_steps("ingest", "extract").build(name="docs_to_entit
result   = ExecutionEngine().execute_pipeline(pipeline)
```

Components: `PipelineBuilder`, `Pipeline`, `ExecutionEngine`, `FailureHandler`, `PipelineValidator`, `ParallelismManager`, `ResourceScheduler`

Explorer

FastAPI Knowledge Explorer with Ontology Hub, WebSocket progress, bidirectional path finding, and indexed search (0.004ms on 118k nodes).

```
semantica-explorer --graph my_graph.json
```

Routes: graph, ontology, provenance, decisions, analytics, SPARQL, temporal, annotations, export/import, vocabulary

Utilities

LLM Providers

Unified interface to all supported LLM providers.

```
from semantica.llms import Groq, OpenAI, LiteLLM
import os

llm = Groq(model="llama-3.3-70b-versatile", api_key=os.getenv("GROQ_API_KEY"))
llm = OpenAI(model="gpt-4o", api_key=os.getenv("OPENAI_API_KEY"))
# Anthropic, Gemini, Ollama, DeepSeek via LiteLLM:
llm = LiteLLM(model="anthropic/claude-opus-4-7", api_key=os.getenv("ANTHROPIC_A
```

Supported providers: OpenAI, Anthropic, Google Gemini, Groq, Ollama, DeepSeek, Novita AI, HuggingFace, plus LiteLLM (100+ models via one interface)

MCP Server

Exposes Semantica as an MCP stdio server for IDE and agent integrations.

```
python -m semantica.mcp_server
```

Integrations: Claude Desktop, VS Code, Cursor, Windsurf, Cline. 15 MCP tools are exposed.

Seed

Bootstrap knowledge graphs from verified structured sources: fixed-point reference data, controlled vocabularies, and domain anchors.

```
from semantica.seed import SeedDataManager

seed = SeedDataManager()

# Load trusted reference data from CSV / JSON / a database / an API
seed_data = seed.load_from_csv("seed_data/industries.csv", entity_type="Industr

# Merge seed data with extraction output (seed values win on conflict by defaul
combined = seed.integrate_with_extracted(
    {"entities": seed_data, "relationships": []},
    {"entities": extracted_entities, "relationships": extracted_relationships},
    merge_strategy="seed_first",
)
```

Use cases: anchoring extraction with known entities, pre-populating ontology classes, deterministic test graph generation.

Evals

Scores decision-intelligence outputs (decision records, audit trails, reasoning text) with a registry of deterministic and model-backed evaluators plus a small run harness.

```
from semantica.evals import evaluate, list_evaluators

list_evaluators()
# ['decision_scores', 'exact_match', 'keyword_check', 'length_range',
#  'levenshtein', 'llm_as_judge', 'numeric_range', 'regex_match', 'rouge',
#  'temporal_range']

cases = [("apple", "aple"), ("night", "nacht")]
summary = evaluate(cases, evaluators=["levenshtein"])
print(summary.total, summary.passed, summary.pass_rate)
```

Public API: `evaluate(cases, evaluators, config=None)`, `list_evaluators()`, `get_evaluator(name)`, and the `EvalMetric` / `CaseResult` / `EvalSummary` result types. See the Evals reference.

Core

Base classes, shared data models, and the plugin registry used across all modules.

```
from semantica.core import Semantica, PluginRegistry, ConfigManager

# ConfigManager loads a Config; Config.get() does dotted lookups
config = ConfigManager().load_from_file("config.yaml")
batch  = config.get("processing.batch_size", default=32)

# Top-level orchestrator: pass the Config object (or a dict), not a path
sem = Semantica(config=config)
sem.initialize()

# Plugin registry: register custom components under a name
registry = PluginRegistry()
registry.register_plugin("my_ingestor", MyCustomIngestor, version="1.0.0")
```

Components: `Semantica`, `PluginRegistry`, `ConfigManager`, `Config`, `LifecycleManager`, `HealthStatus`, `MethodRegistry`

Utils

Shared utilities for ID generation, date parsing, validation, and logging.

```
from semantica.utils import helpers, validators, logging
```

Components: `helpers`, `validators`, `constants`, `types`, `exceptions`, `logging`, `ProgressTracker`

Common Module Chains

Document → KG | GraphRAG | AI Agent | Compliance Pipeline | Web Scraping → Graph | Temp

Load documents from any source and turn them into a queryable knowledge graph.

Pipeline: `Ingest` → `Parse` → `Normalize` → `Semantic Extract` → `GraphBuilder` → `KG`

```
from semantica.ingest import FileIngestor
from semantica.parse import DocumentParser
from semantica.semantic_extract import NERExtractor, RelationExtractor
from semantica.kg import GraphBuilder

sources       = FileIngestor().ingest("data/")
text          = DocumentParser().parse(sources[0].path)["full_text"]
ner           = NERExtractor(method="llm", provider="groq", llm_model="llama-3.
rel           = RelationExtractor(method="llm", provider="groq", llm_model="lla
entities      = ner.extract(text)
relationships = rel.extract(text, entities=entities)
graph         = GraphBuilder(merge_entities=True).build(
                    {"entities": entities, "relationships": relationships}
                )
```

Best for: research pipelines, enterprise data extraction, document intelligence

Module Index

MODULE	PURPOSE	KEY CLASSES
ingest	Data ingestion	`FileIngestor`, `WebIngestor`, `ParquetIngestor`, `XMLIngestor`
parse	Document parsing	`DocumentParser`, `DoclingParser`
split	Text chunking	`TextSplitter`
normalize	Data cleaning	`TextNormalizer`, `EntityNormalizer`, `LanguageDetector`
semantic_extract	NER & relation extraction	`NERExtractor`, `RelationExtractor`, `TripletExtractor`, `SemanticAnalyzer`, `SemanticNetworkExtractor`, `ExtractionValidator`
kg	Graph construction	`GraphBuilder`, `TemporalGraphQuery`, `SimilarityCalculator`
ontology	Schema management	`OntologyGenerator`, `SHACLGenerator`
reasoning	Logical inference	`Reasoner`, `DatalogReasoner`
embeddings	Vector embeddings	`EmbeddingGenerator`
vector_store	Vector database	`VectorStore`
graph_store	Graph database	`GraphStore`
triplet_store	RDF triple store	`TripletStore`
deduplication	Entity resolution	`DuplicateDetector`, `EntityMerger`, `ClusterBuilder`, `MergeStrategyManager`
conflicts	Conflict resolution	`ConflictDetector`, `ConflictResolver`, `SourceTracker`
context	Agent context & decisions	`AgentContext`, `ContextGraph`
provenance	W3C PROV-O lineage	`ProvenanceManager`
change_management	Version control	`TemporalVersionManager`
export	Data export	`RDFExporter`, `ParquetExporter`
visualization	Graph visualization	`KGVisualizer`
pipeline	Workflow orchestration	`Pipeline`, `PipelineBuilder`
explorer	Knowledge Explorer UI	`semantica-explorer --graph <file>`
llms	LLM providers	`Groq`, `OpenAI`, `create_provider`
mcp_server	MCP stdio server	`python -m semantica.mcp_server`
seed	KG bootstrapping from structured sources	`SeedDataManager`
evals	Decision-intelligence evaluation	`evaluate`, `list_evaluators`, `EvalSummary`
core	Base classes & registry	`Semantica`, `ConfigManager`, `PluginRegistry`, `LifecycleManager`
utils	Shared utilities	`helpers`, `validators`

- **Getting Started**: your first knowledge graph in 5 minutes.
- **Cookbook**: 40+ domain notebooks with real-world examples.
- **API Reference**: full technical documentation.

‹ Core Concepts

Choose the Right Module ›

Powered by mintlify